

ScriptLens — Testing Process

Conventional and unconventional testing methods for the v3 structure engine

1. Context and Scope

Plot-contradiction detection is out of the v3 product scope. Testing therefore focuses on the structure engine: scene parsing, the scene dependency graph, orphan-scene detection, simulate cut / edit, scene function impact (setup and payoff), and the draft workflow.

Standard software tests check syntax, HTTP status codes, and database integrity. ScriptLens operates on story logic: it translates fuzzy natural language (Fountain / PDF text) into a deterministic directed acyclic graph. Conventional tests alone miss semantic drift, structural over-sensitivity, and false-positive cascades, so the process below pairs a conventional backbone with targeted unconventional methods.

In scope for testing:

- Fountain / PDF ingest and scene parsing
- Scene dependency graph (continuity and causal edges)
- Orphan-scene detection (hard orphans and loose chains)
- Simulate cut (delete impact) and simulate edit (edge delta)
- Scene function impact (plant / payoff / setup roles)
- Draft workflow (delete, apply edit, undo, export) and the API

2. Conventional Testing (the backbone)

These provide deterministic, repeatable coverage and form the CI gate.

2.1 Golden-file structure corpus

Hand-authored micro-scripts (5-10 scenes), each paired with a YAML ground-truth file that asserts the expected structural outputs.

- Expected orphan scenes (with hard vs. loose-chain type)
- Expected dependency edges (or edge-count bounds)
- Expected simulate-cut impacted scenes and risk tier
- Expected simulate-edit edge delta (added / removed / changed)
- Expected scene-function roles (plant, payoff, setup)

Scored with precision / recall per capability, mirroring the existing baseline scorer. This is the direct successor to the retired planted-contradiction corpus.

2.2 Wire up existing but unused ground truth

The demo ground truth already defines expected orphans and expected simulate-edit results, but only simulate-delete is evaluated today. Wiring the remaining sections is free coverage.

2.3 Unit and regression tests

- Per-detector unit tests for parser, graph, and orphan logic
- Golden-output regression tests on stable fixtures
- Deterministic seeds so semantic embeddings stay reproducible

2.4 False-positive (clean) corpus

Run produced, professionally written screenplays that contain no planted problems. Re-point pass / fail

from contradiction false positives to orphan false positives and spurious high-risk flags.

2.5 API contract tests

Cover the full request path end to end: upload, scripts, orphans, orphan-graph, simulate/cut, simulate/edit, draft/delete, draft/apply-edit, draft/undo, and draft/export - with happy-path and malformed input cases.

3. Unconventional Testing (high return on investment)

Domain-specific methods that stress story logic and NLP resilience in ways ordinary unit tests cannot.

3.1 Metamorphic - entity-swap isomorphism

Rename an entity consistently across a script (for example ALICE to CHARACTER_X, GUN to OBJECT_Y). The dependency graph and orphan set must stay unchanged. Any difference exposes name, casing, or state leakage in the NLP layer. Cheapest, most deterministic, highest-value test for a structure engine; allow a small tolerance for legitimate name collisions.

3.2 Metamorphic - scene-permutation locality

Swap two adjacent scenes that share no entities. The rest of the graph must remain identical, verifying that a local edit produces only a local change.

3.3 Synthetic Chekhov's-gun generator

Programmatically generate three-scene plant -> filler -> payoff scripts across naming and formatting variants ("a brass key", "the key", "it"; uppercase, lowercase, buried in dialogue). Assert the setup-payoff edge and the correct scene-function role resolve every time. Isolates fuzzy NLP behavior in millisecond tests instead of debugging it inside full-length PDFs.

3.4 Graceful degradation (garbage-in) curve

Inject OCR noise, missing punctuation, and lowercased sluglines at 5%, 10%, 25%, and 50%. Orphan counts should drift gradually and the engine should downgrade from full to limited structure mode rather than crash or produce wildly unstable output.

3.5 Narrative chaos - single-word sensitivity

Delete one noun or named entity at a time and measure the change in graph edges. Use it as an outlier detector: flag hyper-sensitive nodes (one word removal un-links many scenes) and under-sensitive nodes (deleting a major action block changes nothing).

3.6 Deferred methods

- Human-vs-engine correlation (blind readers ranking vital scenes, Spearman rho = 0.75): valuable for trust, but expensive; revisit once the structure corpus exists.
- Client memory / DOM-thrashing benchmarks: not applicable until a browser client UI exists (the current product is a Python engine plus API).

4. Summary Matrix

Golden-file structure corpus	Conventional	Orphans, cut/edit, functions	Core precision/recall CI gate
------------------------------	--------------	------------------------------	-------------------------------

Existing-YAML wiring	Conventional	Orphans, simulate edit	Free coverage already defined
Unit / regression	Conventional	Parser, graph, orphan logic	Fast per-component safety net
False-positive corpus	Conventional	Clean produced scripts	Guards against over-flagging
API contract tests	Conventional	All endpoints	Protects the public surface
Entity-swap isomorphism	Unconventional	NLP / entity extraction	Guarantees naming neutrality
Scene-permutation locality	Unconventional	Dependency graph	Confirms edits stay local
Chekhov generator	Unconventional	Edge-resolution logic	Isolates NLP edge cases fast
Graceful degradation curve	Unconventional	Ingest resilience	Ensures no catastrophic failure
Single-word sensitivity	Unconventional	Graph sensitivity	Finds brittle / dead nodes

5. Recommended Sequence

1. Decouple CI from contradiction detection (done).
2. Archive the contradiction corpus and assets reversibly.
3. Design the structure ground-truth schema (orphans, edges, cut, edit, functions) with a reusable template.
4. Seed 15-20 micro-scripts, reusing the orphan-spec and simulate demos.
5. Write a structure baseline scorer and set a new CI gate on structure metrics.
6. Add the two highest-value metamorphic tests (entity-swap isomorphism and the Chekhov generator); they need no corpus.

6. What Needs To Be Done - In Plain English

Here is the whole plan without the jargon. Right now most of our automated checks test a feature we are removing (spotting story contradictions). We need checks that test the features we are keeping: loose scenes, what breaks when you cut or rewrite a scene, and whether a setup earlier in the script still pays off later. The steps below get us there.

Step 1 - Stop failing builds over the old feature (done)

Our automated pipeline used to block all work if the old contradiction feature dipped in accuracy. We have switched that off, so the team is no longer blocked by a feature we are retiring. The old check can still be run by hand when someone wants it.

Step 2 - Put the old test material aside safely

Move the old contradiction scripts and answer keys into a clearly labelled 'legacy' area. We are not deleting anything, just getting it out of the way so it does not confuse the new work. If we ever bring the feature back, it is all still there.

Step 3 - Write down the 'right answers' for a few examples

For a small set of example scripts, agree in a simple checklist what the correct result should be: which scenes are loose, which later scenes break if you remove a given scene, and which early setups should connect to later payoffs. This checklist is what we measure the tool against.

Step 4 - Build about 15 to 20 small example scripts

Write short, deliberate example scripts (five to ten scenes each) that each contain a known situation, and pair every one with its checklist of right answers. We can reuse the demo scripts we already have as a starting point. Quality matters more than quantity.

Step 5 - Build a simple scorekeeper

Create a small program that runs the tool on every example script, compares what it found against the right answers, and reports a clear score (how much it got right, and how often it raised a false alarm). Wire this score into the automated pipeline as the new pass/fail gate.

Step 6 - Add two clever safety checks

- Rename test: rename every character and object in a script, run it again, and confirm the tool's results do not change. If they do, the tool is unfairly reacting to specific names.
- Planted-clue test: automatically create tiny scripts where a clue is introduced early and used later, and confirm the tool always connects the two, no matter how the clue is worded.

Where to start now

Step 1 is complete. The best next actions are Step 3 (agree the checklist format for right answers) and Step 6's rename test, because neither needs the full example library to exist first and both deliver value immediately.